Understanding Your Dataset with Pandas – Structured Summary

1. How Big is the Data?

- Check the number of rows and columns.
- Helps understand the dataset size.

Syntax:

df.shape

---

2. How Does the Data Look?

- View the first few records or random samples.
- sample() is preferred because it avoids bias if the data is sorted.

Syntax:

df.head()

df.sample(5)

---

3. What are the Data Types?

- Check the data type of each column.
- Shows memory usage and missing values.

Syntax:

df.info()

---

4. Are There Missing Values?

- Count missing values in each column.
- Helps decide whether to remove or fill missing data.

Syntax:

df.isnull().sum()

---

5. How Does the Data Look Mathematically?

- Generates statistical summary for numerical columns.
- Shows count, mean, standard deviation, minimum, maximum, and percentiles.

Syntax:

df.describe()

---

6. Are There Duplicate Values?

- Counts duplicate records in the dataset.
- Duplicate data can affect model performance.

Syntax:

df.duplicated().sum()

---

7. What is the Correlation Between Columns?

- Measures the relationship between numerical columns.
- Helps identify important features for prediction.

Syntax:

df.corr()

Common Pandas Functions

Function	Purpose
df.shape	Number of rows and columns
df.head()	First 5 rows
df.sample(5)	Random 5 rows
df.info()	Data types and memory usage
df.isnull().sum()	Missing values count
df.describe()	Statistical summary
df.duplicated().sum()	Duplicate rows count
df.corr()	Correlation between columns

Key Takeaways

- df.shape → Check dataset size.
- df.head() / df.sample() → Preview data.
- df.info() → View data types and memory usage.
- df.isnull().sum() → Find missing values.
- df.describe() → Get statistical summary.
- df.duplicated().sum() → Detect duplicate records.

- df.corr() → Analyze relationships between numerical features.

Exploratory Data Analysis (EDA) – Univariate Analysis (Structured Summary)

1. What is Univariate Analysis?

- Univariate Analysis is a part of **Exploratory Data Analysis (EDA)**.
- It analyzes **one variable (one column) at a time**.
- Helps understand the distribution, trends, and characteristics of individual features.

---

2. Types of Data

Numerical Data

- Contains quantitative values.
- Examples: Age, Height, Weight, Salary, Price.

Categorical Data

- Contains qualitative values or categories.
- Examples: Gender, Country, Class, Department.

---

3. Analyzing Categorical Data

Count Plot

- Shows the frequency of each category.
- Best for comparing category counts.

Syntax:

```
import seaborn as sns


sns.countplot(x='column_name', data=df)
```

Pie Chart

- Shows the percentage distribution of categories.

Syntax:

```
df['column_name'].value_counts().plot(kind='pie', autopct='%1.1f%%')
```

---

4. Analyzing Numerical Data

Histogram

- Shows the distribution of numerical values using bins.

Syntax:

```
import matplotlib.pyplot as plt


plt.hist(df['column_name'], bins=10)
plt.show()
```

Distplot / KDE Plot

- Displays the data distribution with a smooth probability density curve.

Syntax:

```
sns.histplot(df['column_name'], kde=True)
```

Box Plot

- Displays the five-number summary:
    - Minimum
    - First Quartile (Q1)
    - Median
    - Third Quartile (Q3)
    - Maximum
- Helps identify outliers.

Syntax:

```
sns.boxplot(x=df['column_name'])
```

---

5. Statistical Summary

Mean

```
df['column_name'].mean()
```

Maximum

```
df['column_name'].max()
```

Minimum

```
df['column_name'].min()
```

Median

```
df['column_name'].median()
```

Skewness

```
df['column_name'].skew()
```

Complete Statistical Summary

df['column_name'].describe()

Common Visualization Methods

Visualization	Used For
Count Plot	Frequency of categories
Pie Chart	Percentage distribution
Histogram	Distribution of numerical data
KDE Plot	Probability density distribution
Box Plot	Detecting outliers and spread

Exploratory Data Analysis (EDA) – Bivariate & Multivariate Analysis

1. Bivariate Analysis

- Studies the relationship between **two variables**.
- Example: Relationship between **Age** and **Salary**.

2. Multivariate Analysis

- Studies the relationship between **three or more variables**.
- Example: Relationship between **Age**, **Salary**, and **Department**.

---

1. Numerical vs Numerical Analysis

Scatter Plot

- Used to find the relationship between two numerical variables.

Example: Relationship between **Age** and **Salary**.

Syntax:

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.scatterplot(x='Age', y='Salary', data=df)
plt.show()
```

With Third Variable (Multivariate)

- Color points based on Gender.

```
sns.scatterplot(x='Age', y='Salary', hue='Gender', data=df)
plt.show()
```

Line Plot

- Used to show trends over time.

Example: Monthly Sales.

Syntax:

```
sns.lineplot(x='Month', y='Sales', data=df)
plt.show()
```

---

2. Numerical vs Categorical Analysis

Bar Plot

- Compares average numerical values across categories.

Example: Average Salary in each Department.

Syntax:

```
sns.barplot(x='Department', y='Salary', data=df)
plt.show()
```

Box Plot

- Shows distribution and detects outliers.

Example: Salary distribution by Department.

Syntax:

```
sns.boxplot(x='Department', y='Salary', data=df)
plt.show()
```

Distribution Plot (Histogram + KDE)

- Compares distribution of numerical values.

Example: Age distribution by Gender.

Syntax:

```
sns.histplot(data=df, x='Age', hue='Gender', kde=True)
plt.show()
```

---

3. Categorical vs Categorical Analysis

Heatmap

- Shows relationship between two categorical variables.

Example: Gender vs Department.

Syntax:

```
import pandas as pd

table = pd.crosstab(df['Gender'], df['Department'])

sns.heatmap(table, annot=True, cmap='Blues')
plt.show()
```

Cluster Map

- Groups similar categories.

Example: Gender vs Department.

Syntax:

```
table = pd.crosstab(df['Gender'], df['Department'])

sns.clustermap(table)
```

---

4. Multivariate Analysis

Pair Plot

- Shows relationships between all numerical columns.

Example: Age, Salary, Experience.

Syntax:

```
sns.pairplot(df)
plt.show()
```

With Category

```
sns.pairplot(df, hue='Department')
plt.show()
```

Examples

Scatter Plot

Age ----------> Salary

25 30000

30 45000

35 60000

Shows whether salary increases with age.

Line Plot

Month → Jan Feb Mar Apr

Sales → 100 150 220 300

Shows sales trend over time.

Bar Plot

Department

IT ████████ 70000

HR ██████ 50000

Sales ████████ 60000

Compares average salaries.

Box Plot

IT Department Salary

```
|----|====|====|----|
     ↑
   Median
```

• = Outlier

Shows median, quartiles, and outliers.

Heatmap

IT HR

Male 40 15

Female 20 30

Darker colors represent higher counts.

Pair Plot

Age ↔ Salary

Age ↔ Experience

Salary ↔ Experience

Displays relationships among all numerical variables.

Summary Table

Plot	Variable Type	Purpose	Example
Scatter Plot	Numerical vs Numerical	Relationship	Age vs Salary
Line Plot	Numerical vs Numerical	Trend over time	Month vs Sales
Bar Plot	Numerical vs Categorical	Compare averages	Department vs Salary
Box Plot	Numerical vs Categorical	Distribution & Outliers	Department vs Salary
Histogram/KDE	Numerical vs Categorical	Compare distributions	Age by Gender
Heatmap	Categorical vs Categorical	Relationship between categories	Gender vs Department
Cluster Map	Categorical vs Categorical	Group similar categories	Gender vs Department
Pair Plot	Multivariate	Relationship among multiple numerical variables	Age, Salary, Experience

Feature Scaling – Standardization (Structured Summary)

1. What is Feature Scaling?

- Feature Scaling is the process of bringing numerical features to a **similar scale**.
- It prevents features with large values from dominating those with smaller values.
- Improves the performance of many machine learning algorithms.

Example:

Age	Salary
25	50,000
30	80,000

Here, **Salary** has much larger values than **Age**, so scaling is needed.

---

2. What is Standardization?

- Standardization transforms data so that:
    - **Mean = 0**
    - **Standard Deviation = 1**

Formula

$$Z = \frac{X - \mu}{\sigma}$$

Where:

- **X** = Original value
- **μ** = Mean
- **σ** = Standard deviation
- **Z** = Standardized value

---

3. How Standardization Works

Mean Centering

- Shifts data so that the mean becomes **0**.

Scaling

- Divides each value by the standard deviation.
- Makes all features have similar variance.

---

4. Effect of Outliers

- Standardization **does not remove outliers**.
- Outliers still affect the mean and standard deviation.
- Handle outliers before or separately from scaling.

---

5. Implementation using Scikit-Learn

Import

```
from sklearn.preprocessing import StandardScaler
```

Create Scaler

```
scaler = StandardScaler()
```

Fit and Transform Training Data

```
X_train_scaled = scaler.fit_transform(X_train)
```

Transform Test Data

```
X_test_scaled = scaler.transform(X_test)
```

Best Practice: Use fit_transform() only on the training data. Use transform() on the test data to avoid data leakage.

---

6. Example

Before Scaling

Age	Salary
20	30000
30	60000
40	90000

After Standardization

Age	Salary
-1.22	-1.22
0.00	0.00
1.22	1.22

---

7. When to Use Standardization

Use Standardization for:

- **K-Nearest Neighbors (KNN)**
- **K-Means Clustering**
- **Principal Component Analysis (PCA)**
- **Logistic Regression**
- **Linear Regression (Gradient Descent)**
- **Support Vector Machine (SVM)**
- **Neural Networks**
- **Deep Learning Models**

---

8. When Standardization is NOT Required

Standardization is generally **not needed** for tree-based algorithms because they split data based on feature values rather than distances.

Examples:

- **Decision Tree**
- **Random Forest**
- **XGBoost**
- **LightGBM**
- **CatBoost**

Summary Table

Topic	**Description**
Feature Scaling	Brings features to a similar scale
Standardization	Mean = 0, Standard Deviation = 1
Formula	$Z = \frac{X - \mu}{\sigma}$
Removes Outliers?	❌ No
Best Practice	Fit on training data, transform both training and test data
Used For	KNN, K-Means, PCA, Logistic Regression, SVM, Neural Networks

Topic	Description
Not Required For	Decision Tree, Random Forest, XGBoost, LightGBM, CatBoost

---

Feature Scaling – Normalization (Structured Summary)

1. What is Normalization?

- Normalization is a feature scaling technique that transforms numerical values into a **fixed range**.
- It prevents features with larger values from dominating smaller ones.
- Commonly used for **distance-based algorithms** and **image processing**.

Example (Before Normalization)

Age	Salary
20	30000
30	60000
40	90000

---

2. Min-Max Scaling

- Rescales data to a range between **0 and 1**.
- Most commonly used normalization technique.

Formula

$$X_{new} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Where:

- **X** = Original value
- **Xmin** = Minimum value
- **Xmax** = Maximum value

Example

Original	Normalized
20	0.0
30	0.5

Original	Normalized
40	1.0

Syntax

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)
```

---

3. Mean Normalization

- Centers the data around **0**.
- Values usually lie between **-1 and 1**.
- Less commonly used than Min-Max Scaling.

Formula

$$X_{new} = \frac{X - \bar{X}}{X_{max} - X_{min}}$$

Where:

- $\bar{X}$ = Mean

Example

Original	Mean Normalized
20	-0.5
30	0.0
40	0.5

Syntax (Manual)

```
X_mean = X.mean()

X_scaled = (X - X_mean) / (X.max() - X.min())
```

---

4. Max-Abs Scaling

- Divides every value by the **maximum absolute value**.
- Preserves zero values.
- Best for **sparse datasets**.

Formula

$$X_{new} = \frac{X}{| X_{max} |}$$

Syntax

```
from sklearn.preprocessing import MaxAbsScaler

scaler = MaxAbsScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)
```

---

5. Robust Scaling

- Uses **Median** and **Interquartile Range (IQR)**.
- Less affected by outliers.

Formula

$$X_{new} = \frac{X - \text{Median}}{IQR}$$

Where:

$$IQR = Q3 - Q1$$

Syntax

```
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()

X_train_scaled = scaler.fit_transform(X_train)
```

X_test_scaled = scaler.transform(X_test)

Normalization vs Standardization

Feature	Normalization	Standardization
Output Range	Usually 0 to 1	Mean = 0, Std = 1
Formula	Uses Min & Max	Uses Mean & Standard Deviation
Outlier Sensitive	Yes	Yes
Best For	Image Processing, KNN	Logistic Regression, SVM, PCA, Neural Networks

---

When to Use Normalization

- **K-Nearest Neighbors (KNN)**
- **K-Means Clustering**
- **Image Processing**
- **Neural Networks (sometimes)**
- When feature values have **known minimum and maximum limits**

When to Use Standardization

- **Logistic Regression**
- **Linear Regression (Gradient Descent)**
- **Support Vector Machine (SVM)**
- **Principal Component Analysis (PCA)**
- **Neural Networks**
- When data follows a **normal (Gaussian) distribution**

Summary Table

Technique	Formula	Best Use Case
Min-Max Scaling	$(X - X_{min})/(X_{max} - X_{min})$	General normalization, image data
Mean Normalization	$(X - \bar{X})/(X_{max} - X_{min})$	Centering data around zero

Technique	Formula	Best Use Case
Max-Abs Scaling	(X/	X_{max}
Robust Scaling	$(X - \text{Median})/IQR$	Data with many outliers

---

Machine Learning Pipeline – Structured Summary

1. What is a Pipeline?

A **Pipeline** in Scikit-learn connects multiple ML steps into one workflow.

Typical flow:

Raw Data

↓

Missing Value Handling

↓

Feature Scaling

↓

Categorical Encoding

↓

Model Training

↓

Prediction

Instead of performing every step manually, the Pipeline automates the process.

---

2. Why Use Pipelines?

Automation

Combines:

- Data cleaning
- Feature engineering
- Encoding
- Scaling
- Model training

Data Leakage Prevention

The pipeline ensures that preprocessing is **learned only from training data**.

For example:

StandardScaler()

calculates mean and standard deviation from the training data, not from the test data.

Production Ready

The same preprocessing used during training is automatically applied when making predictions on new data.

---

3. Important Tools

Tool	Purpose
SimpleImputer	Handle missing values
StandardScaler	Scale numerical features
OneHotEncoder	Encode nominal data
OrdinalEncoder	Encode ordinal data
ColumnTransformer	Apply different preprocessing to different columns
Pipeline	Connect all steps together

---

4. Without Pipeline

Normally, you might manually do:

X_train = imputer.fit_transform(X_train)

X_train = scaler.fit_transform(X_train)

X_train = encoder.fit_transform(X_train)

model.fit(X_train, y_train)

Then you have to remember to perform the **exact same transformations** on test/new data.

This can cause:

- More code

- More mistakes
- Data leakage
- Production problems

---

5. Numerical Pipeline

For numerical columns:

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())
])
```

Flow

Missing Values

↓

Mean Imputation

↓

Standard Scaling

---

6. Nominal Pipeline

For nominal categorical columns:

```
nominal_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(drop='first'))
])
```

Flow

Missing Values

↓

Most Frequent Value

↓

One-Hot Encoding

---

7. Ordinal Pipeline

For ordinal categorical columns:

```
ordinal_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OrdinalEncoder(
        categories=[
            ['Bachelor', 'Master', 'PhD'],
            ['Low', 'Medium', 'High']
        ]
    ))
])
```

Flow

Missing Values

↓

Most Frequent Value

↓

Ordinal Encoding

---

8. Combine Using ColumnTransformer

```
from sklearn.compose import ColumnTransformer


preprocessor = ColumnTransformer([
    ('num', num_pipeline, numerical_columns),
    ('ordinal', ordinal_pipeline, ordinal_columns),
    ('nominal', nominal_pipeline, nominal_columns)
])
```

Now different columns automatically go through their appropriate pipelines.

---

9. Final ML Pipeline

Suppose we are using Random Forest:

```
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier


model_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', RandomForestClassifier())
])
```

Train:

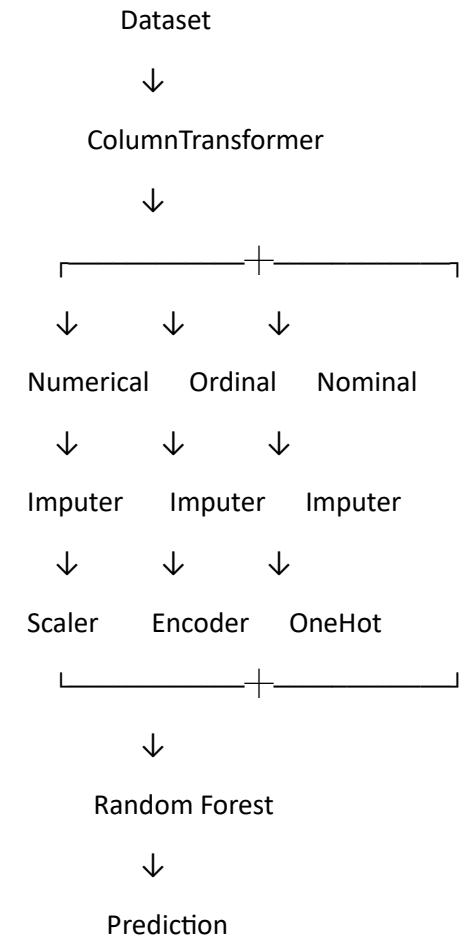
```
model_pipeline.fit(X_train, y_train)
```

Predict:

```
y_pred = model_pipeline.predict(X_test)
```

---

10. Complete Flow



Key Takeaways

- **Pipeline** = connects preprocessing + model.
- **ColumnTransformer** = sends different columns to different preprocessing methods.
- **SimpleImputer** = handles missing values.
- **StandardScaler** = scales numerical features.
- **OrdinalEncoder** = handles ordered categories.
- **OneHotEncoder** = handles nominal categories.
- Pipelines reduce code, prevent **data leakage**, and make ML systems **production-ready**.

Voice

This video provides an introduction to **Function Transformers** in Machine Learning, which are used to transform data into a more normal distribution, helping models perform better. Here are the key notes:

Types of Transformers

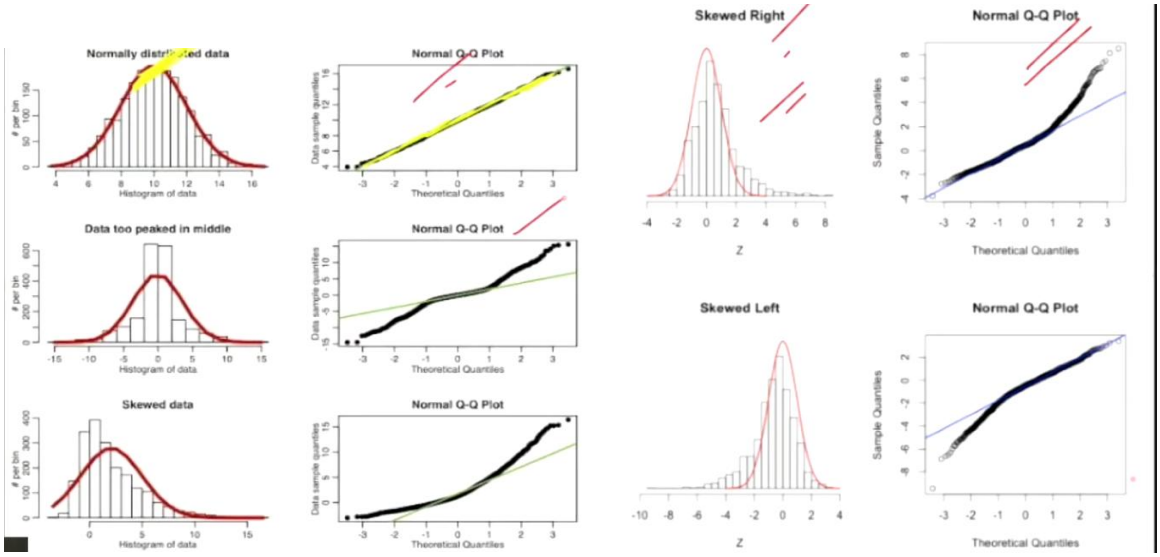
- **Function Transformer:** Uses mathematical functions (Log, Reciprocal, Square/Square Root, Custom).
- **Power Transformer:** Includes *Box-Cox* and *Yeo-Johnson* methods (0:36-1:55).
- **Quantile Transformer:** (0:43).

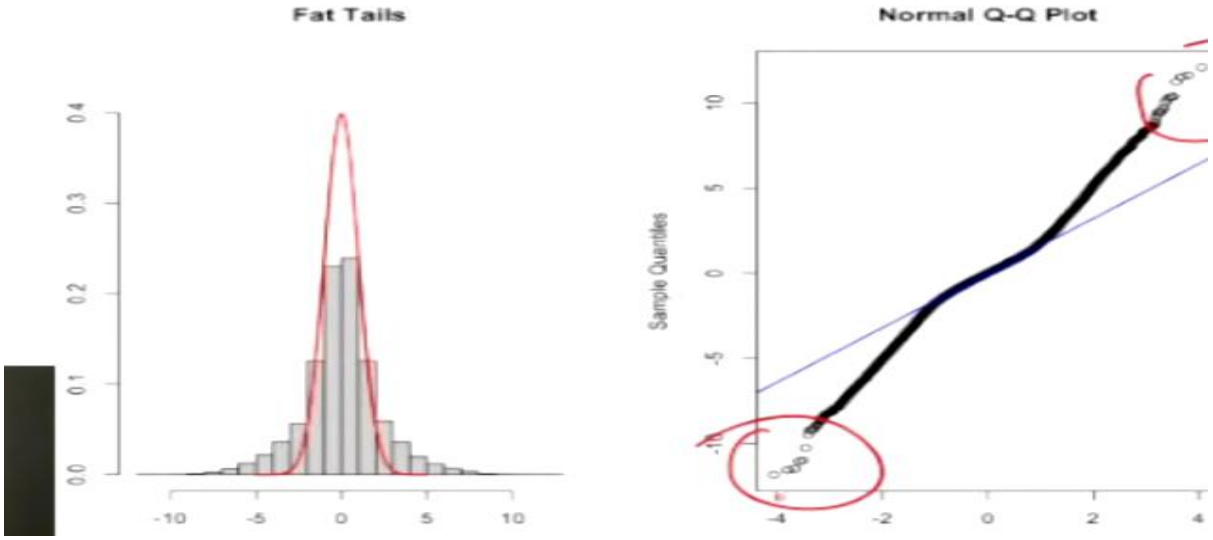
Data Distribution & Q-Q Plots

- Understanding distribution is crucial for linear models like *Linear* and *Logistic Regression* (3:53-4:04).
- **Q-Q Plots (Quantile-Quantile plots):** The most accurate visual tool to compare your data against a normal distribution (5:24-5:56).
- The video describes common distribution issues: **Right-skewed** (long tail on the right), **Left-skewed** (long tail on the left), and data with **Fat/Thin tails** (8:24-9:38).

Transformation Methods

- **Log Transformation:** Great for right-skewed data. It helps convert non-linear relationships to linear and multiplicative relationships into additive ones (10:07-16:40).
- **Reciprocal Transformation:** Used to compress large values (16:40-18:04).
- **Square/Square Root Transformations:** Used for specific compression needs, though square transformations can sometimes create new outliers (18:07-20:01).





Numerical Data Encoding — Structured Summary

1. What is Numerical Encoding?

Numerical encoding transforms **continuous numerical data** into simpler forms such as **categories or binary values**.

Main techniques:

- **Discretization (Binning)**
- **Binarization**

2. Discretization / Binning

Discretization converts continuous values into **bins (categories)**.

Example:

Age: 12, 25, 38, 52, 67, 80

Bins:

0–20 → Young

21–40 → Adult

41–60 → Middle Age

61+ → Senior

Why use it?

- Handles outliers
- Reduces complexity
- Converts numerical data into categories
- Can improve some models

3. Equal Width Binning

Divides the **minimum-to-maximum range into equal-sized intervals**.

Example:

Age: 10 to 70

Bin 1: 10–25

Bin 2: 25–40

Bin 3: 40–55

Bin 4: 55–70

Syntax

```
from sklearn.preprocessing import KBinsDiscretizer

encoder = KBinsDiscretizer(
    n_bins=4,
    encode='ordinal',
    strategy='uniform'
)

X_new = encoder.fit_transform(X)
```

strategy='uniform' → Equal Width Binning.

4. Equal Frequency Binning

Also called **Quantile Binning**.

Each bin contains approximately the **same number of data points**.

Example:

100 students

Bin 1 → 25 students

Bin 2 → 25 students

Bin 3 → 25 students

Bin 4 → 25 students

Syntax

```
encoder = KBinsDiscretizer(
    n_bins=4,
    encode='ordinal',
    strategy='quantile'
)

X_new = encoder.fit_transform(X)
```

strategy='quantile' → Equal Frequency Binning.

5. K-Means Binning

Uses **K-Means clustering** to decide the bin boundaries.

Useful when data naturally forms **different clusters**.

Syntax

```
encoder = KBinsDiscretizer(
    n_bins=4,
    encode='ordinal',
    strategy='kmeans'
)

X_new = encoder.fit_transform(X)
```

6. Comparison of Binning Methods

Method	How it works	Best use
Equal Width	Equal range size	Simple binning
Equal Frequency	Equal number of samples	Uneven distributions
K-Means	Uses clusters	Naturally clustered data

7. Binarization

Binarization converts numerical values into **0 or 1** using a threshold.

Example:

Age > 60 → 1

Age ≤ 60 → 0

Syntax

```
from sklearn.preprocessing import Binarizer

binarizer = Binarizer(threshold=60)

X_new = binarizer.fit_transform(X)
```

Example:

Age = [25, 45, 61, 75]

Result:

[0, 0, 1, 1]

8. Key Difference

Binning:

Continuous → Multiple categories

Example:

Age → Young / Adult / Senior

Binarization:

Continuous → 0 or 1

Example:

Age → 0 / 1

Mixed Type Columns — Structured Summary

1. What are Mixed Type Columns?

A **mixed type column** contains different types of information in the same column.

Example:

Class

S1

S2

A1

B3

Here, the column contains:

- Letters → S, A, B
- Numbers → 1, 2, 3

This can cause problems during ML preprocessing.

2. Problems

One-Hot Encoding

Applying One-Hot Encoding directly can create too many categories.

S1 → column

S2 → column

A1 → column

B3 → column

...

This can lead to:

- High dimensionality
- Sparse data
- Slower training

Ordinal Encoding

Ordinal Encoding assigns numbers:

S1 → 1

S2 → 2

A1 → 3

B3 → 4

The problem is that the model may assume:

B3 > A1 > S2 > S1

But there may be **no actual order** between them.

3. Best Solution

Split the mixed column into separate features.

Example:

Original:

S1

S2

A1

B3

Convert into:

Class Number

S 1

S 2

A 1

B 3

Now each feature has a clear meaning.

4. Using str.extract()

Pandas can extract letters and numbers using regular expressions.

Extract letters

df['Class'] = df['Seat'].str.extract(r'([A-Za-z]+)')

Extract numbers

df['Number'] = df['Seat'].str.extract(r'(\d+)')

Example:

Seat = "S25"

Class → S

Number → 25

5. Using Lambda Functions:

: For more complex formatting, apply lambda functions to rows to search for specific regex patterns or groups, which helps in cleaning and restructuring the data into useful, uniform features

For more complex patterns:

```
df['Class'] = df['Seat'].apply(
    lambda x: x[0] if isinstance(x, str) else None
)
```

Extract number:

```
df['Number'] = df['Seat'].apply(
    lambda x: int(x[1:]) if isinstance(x, str) else None
)
```

Date & Time Feature Engineering

- **Convert date to datetime** using pd.to_datetime().
- Use **.dt** to extract useful features:
    - .dt.day → Day
    - .dt.month → Month
    - .dt.year → Year
    - .dt.day_name() → Day name
    - .dt.quarter → Quarter
    - .dt.hour → Hour
- Create **Weekend** flag:

```
df['Weekend'] = df['Date'].dt.dayofweek >= 5
```

- Calculate **time differences** by subtracting date columns.

Example:

```
df['Date'] = pd.to_datetime(df['Date'])
df['Month'] = df['Date'].dt.month
df['Day'] = df['Date'].dt.day
df['Year'] = df['Date'].dt.year
```

Key idea: Convert dates → Extract meaningful features → Use them for ML.

Complete Case Analysis (CCA)

- **CCA** handles missing data by **removing rows containing missing values**.
- Best used when:
    - Missing data is **MCAR (Missing Completely At Random)**.
    - Missing values are **less than ~5%**.

- **Advantages:** Simple and easy to implement; distribution can remain similar.
- **Disadvantages:** Causes data loss and may introduce bias.
- Always compare the data **before and after CCA** to check whether the distribution has changed.

Example:

df = df.dropna()

Check missing values:

df.isnull().sum()

Key idea: Missing rows → Remove them → Check whether the remaining data is still representative.

---

1. Mean Imputation

Calculate the average:

(20 + 25 + 30 + 35) / 4 = 27.5

Replace NaN with **27.5**.

df['Age'].fillna(df['Age'].mean())

2. Median Imputation

Sorted values:

20, 25, 30, 35

Median = **27.5**

df['Age'].fillna(df['Age'].median())

Median is better when outliers exist.

3. Arbitrary Value

Replace missing value with a fixed value:

20

25

-999

30

35

df['Age'].fillna(-999)

4. End of Distribution

Replace missing value with an extreme value, for example:

Mean + 3 × Standard Deviation

This keeps the missing value clearly separated from normal values.

5. Multivariate Imputation

Instead of looking only at Age, use other columns:

Age Salary Experience

25 30000 2

30 50000 5

NaN 45000 4

The model can use **Salary + Experience** to estimate the missing Age.

Categorical Missing Value Imputation

Categorical missing values are usually handled using **Most Frequent Imputation** or **Missing Category Imputation**.

1. Most Frequent Imputation

Replace missing values with the **most common category (mode)**.

Example:

City

Bangalore

Mumbai

Bangalore

NaN

Bangalore

Most frequent = **Bangalore**

After imputation:

City

Bangalore

Mumbai

Bangalore

Bangalore

Bangalore

```
from sklearn.impute import SimpleImputer


imputer = SimpleImputer(strategy='most_frequent')
```

df[['City']] = imputer.fit_transform(df[['City']])

Best when: Missing values are low (around **5% or less**) and mostly random.

Problem: It can increase the frequency of the most common category and change the original distribution.

2. Missing Category Imputation

Create a new category called **"Missing"**.

Example:

City

Bangalore

Mumbai

Bangalore

NaN

Delhi

After:

City

Bangalore

Mumbai

Bangalore

Missing

Delhi

```
imputer = SimpleImputer(
   strategy='constant',
   fill_value='Missing'
)
```

df[['City']] = imputer.fit_transform(df[['City']])

Best when: A large amount of data is missing or the missingness itself may contain useful information.

Quick Rule

- **Small missing %** → Most Frequent
- **Large missing % / missingness may matter** → Missing Category
- **Numerical data** → Mean / Median / other numerical imputation

- **Categorical data** → Mode / Missing category

This video covers advanced techniques for **handling missing data** in machine learning, building upon previous lessons on univariate data imputation. Here are the key topics discussed:

1. Random Sample Imputation (2:12 - 21:33)

- **Concept:** Instead of using mean, median, or mode, you fill missing values by randomly picking an existing observed value from the same column.
- **Benefits:** It helps preserve the original **data distribution** and variance, which is highly beneficial when using linear models.
- **Drawbacks:** It can alter the covariance (relationship) between variables. It also requires keeping the training set in memory for deployment, making it memory-intensive.

2. Missing Indicator (21:33 - 30:17)

- **Concept:** This involves creating a new binary column (True/False) that flags where the original data was missing.
- **Purpose:** It allows the machine learning model to distinguish between rows where data was present versus rows where it was missing. This can often improve model performance.
- **Implementation:** You can use the dedicated MissingIndicator class or simply set add_indicator=True in Scikit-Learn's SimpleImputer.

3. Automatically Selecting Imputation Parameters (30:17 - 36:36)

- **Concept:** Using GridSearchCV to automatically find the best imputation strategy (e.g., mean vs. median) for your specific dataset.
- **Process:** This is done by creating a machine learning **pipeline** that includes the imputer and the model, allowing you to tune the imputation technique as a hyperparameter.

KNN Imputer

KNN Imputer is a **multivariate missing-value technique** that uses other similar rows to estimate a missing value.

Concept

Think of it like asking **"Which existing data points are most similar to this row?"**

Example:

Age	Salary	Experience
25	30000	2
30	40000	4
28	NaN	3
35	50000	8

For the missing Salary of the person aged 28, KNN looks at other features such as **Age and Experience** and finds the most similar people.

If the nearest 2 people have salaries:

30000

40000

Then:

Estimated Salary = (30000 + 40000) / 2

= 35000

So the missing value becomes **35000**.

How KNN Imputer Works

1. Find rows with similar feature values.
2. Calculate the **distance** between rows.
3. Select the **K nearest neighbors**.
4. Use their values to estimate the missing value.

Syntax

```
from sklearn.impute import KNNImputer


imputer = KNNImputer(n_neighbors=3)


X_new = imputer.fit_transform(X)
```

n_neighbors

KNNImputer(n_neighbors=3)

Means it uses the **3 nearest rows** to calculate the missing value.

Weights

Uniform:

KNNImputer(weights='uniform')

All neighbors have equal importance.

Distance:

KNNImputer(weights='distance')

Closer neighbors have **more importance**.

Advantages

- Uses information from multiple columns.
- Usually better than simple mean/median imputation.
- Preserves relationships between features.

Disadvantages

- Computationally expensive.
- Requires more memory.
- Sensitive to feature scale.

Important

Since KNN uses **distance**, scaling should generally be done before KNN:

```
from sklearn.preprocessing import StandardScaler


scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

Key idea:
Missing value → Find similar rows → Select K nearest neighbors → Use their values → Fill the missing value.